

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Distributed Object Storage & Fault Tolerance Testing
Data 2026/6/24 Class Applied Computing Name Wang Shiyu ID 202383910020

一、Experimental purpose

Master Object Storage: Understand the difference between traditional file paths and S3-compatible Bucket/Object structures.

Experience Fault Tolerance: Observe how data remains accessible even when physical "disks" are destroyed.

二、Experiment Contents

In this experiment, students will simulate a distributed cloud object storage system similar to:

Amazon S3, Alibaba Cloud OSS

using: MinIO, Docker Compose

Students will:

- Deploy a 4-disk distributed MinIO cluster

- Upload and manage cloud objects

- Observe backend storage fragmentation

- Simulate storage node failure

- Verify distributed storage fault tolerance

三、Detailed steps of the experiment

Lab Report Requirements: Clearly present the procedure step by step, with corresponding screenshots provided for each stage. **Screenshots must demonstrate that they were taken on your**

own computer, for example by including visible system time, desktop environment, or other identifiable personal elements.

Step 1: Deploy a 4 - Node Distributed Cluster

In your Codespaces terminal, create a dedicated directory for this lab:

```
mkdir minio - lab && cd minio - lab
```

Step 2: Create Storage Directories

Create four independent storage directories to simulate four storage disks.

```
mkdir -p data1 data2 data3 data4
```

Verify:

```
ls
```

Step 3: Create docker-compose.yml

Create a file named:

docker-compose.yml in minio-lab

with the following content:

```
services:

  minio:
    image: quay.io/minio/minio
    container_name: cloud-minio
```

```
volumes:
  - ./data1:/data1
  - ./data2:/data2
  - ./data3:/data3
  - ./data4:/data4
```

```
ports:
```

- "9000:9000"
- "9001:9001"

environment:

MINIO_ROOT_USER: admin

MINIO_ROOT_PASSWORD: password123

command: server /data{1...4} --console-address
":9001"

mc:

image: minio/mc

container_name: cloud-mc

depends_on:

- minio

entrypoint: /bin/sh

tty: true

Step 4: Explain the docker-compose.yml File

Students should explain the meaning of the following components:

Component	Meaning
image	Docker image used to create the container
container_name	
volumes	
ports	
environment	

Component	Meaning
command	Start MinIO distributed storage service
depends_on	

Step 5: Launch the Distributed Cluster

Start all services:

```
cd minio-lab
```

```
docker compose up -d
```

Verify running containers:

```
docker ps
```

You should see:

```
cloud-minio
```

```
cloud-mc
```

Step 6: Configure MinIO Client Alias

Configure the MinIO client (mc) to connect to the MinIO server.

```
docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
```

Observation:

You should see: Added myminio successfully.

Step 7: Create a Bucket

Create a cloud storage bucket.

```
docker exec cloud-mc mc mb myminio/my-cloud-storage
```

Step 8: Create and Upload a Large Test File

In distributed object storage systems, large files are automatically divided into multiple fragments and distributed across different storage disks. This step demonstrates the Erasure Coding and distributed storage mechanism.

8.1 Create a Large Test File

Generate a 20 MB random binary file:

```
dd if=/dev/urandom of=bigfile.bin bs=1M count=20
```

8.2 Verify the File

Check the file size:

```
ls -lh bigfile.bin
```

8.3 Copy the File into the mc Container

```
docker cp bigfile.bin cloud-mc:/bigfile.bin
```

8.4 Upload the File to Cloud Storage

```
docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
```

Observation:

You should see upload progress information.

Step 9: Verify Object Upload

List all objects in the bucket.

```
docker exec cloud-mc mc ls myminio/my-cloud-storage/
```

You should see: bigfile.bin

Step 10: Observe Backend Storage Fragmentation

Observe how MinIO distributes data across multiple storage disks.

Run:

```
ls -R ./data1
```

Then:

```
ls -R ./data2
```

Then:

```
ls -R ./data3
```

Then:

```
ls -R ./data4
```

Observation:

Students should notice:

- object metadata
- fragmented storage structure
- distributed object placement

Students must compare the structure across all disks

Cloud Object Storage Architecture

Object



Metadata (xl.meta)



Chunk Splitting



Distributed Parts

↓
Multiple Storage Nodes

Step 11: Simulate Disk Failure

Distributed cloud storage systems must tolerate hardware failures.

11.1 Stop the Cluster

`docker compose down`

11.2 Delete One Storage Disk

`sudo rm -rf data3`

`mkdir data3`

This simulates disk failure.

11.3 Restart the Cluster

`docker compose up -d`

Step 12: Verify Fault Tolerance

Check whether objects are still accessible.

`docker exec cloud-mc mc ls myminio/my-cloud-storage/`

Observation:

`bigfile.bin` should still exist.

This demonstrates:

Erasur Coding and distributed redundancy

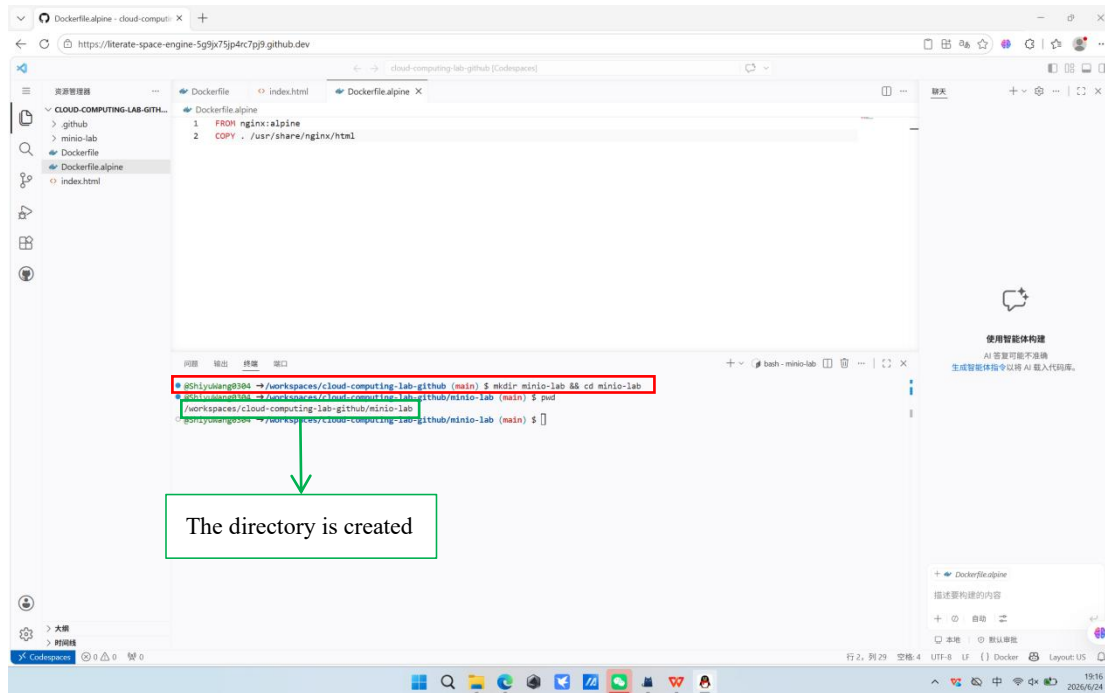
Step 13: Use git to save your changes.

四、 Experiment Process

1. Step 1: Creating the Lab Directory

First, I created a dedicated directory for this MinIO object storage experiment in the Codespaces

terminal: `mkdir minio-lab && cd minio-lab`



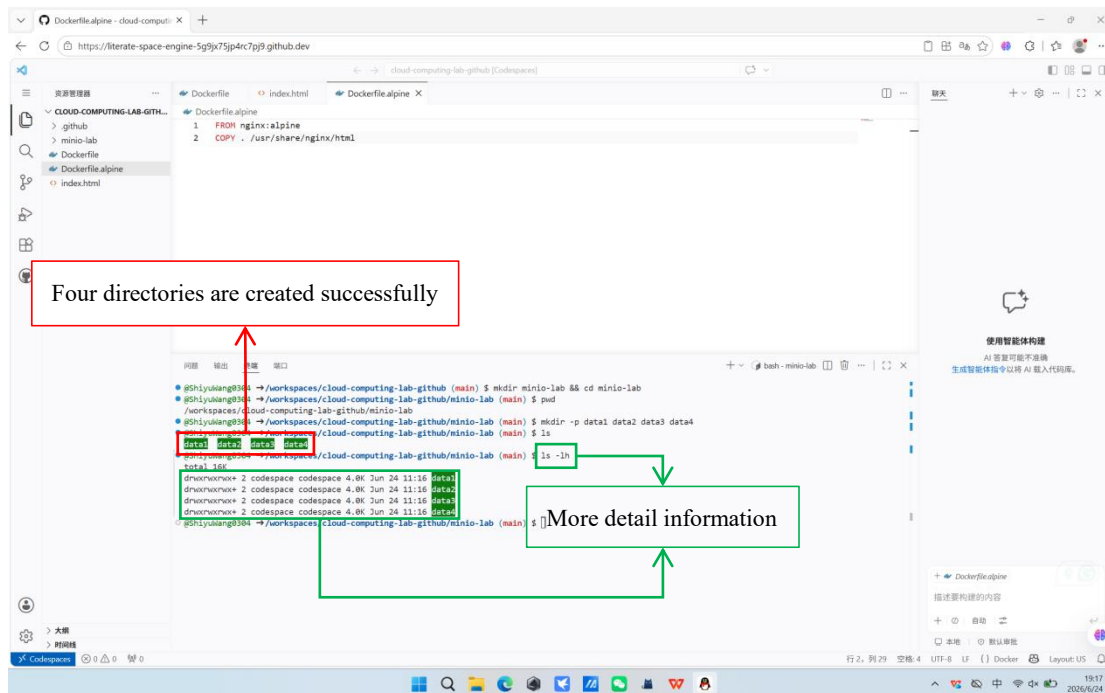
This directory was used to store the Docker Compose configuration file, simulated storage disks, and the test object file.

2. Step 2: Creating Four Storage Directories

Next, I created four independent directories to simulate four physical storage disks: `mkdir -p data1 data2 data3 data4`

Then, I verified that the directories were created successfully: `ls`

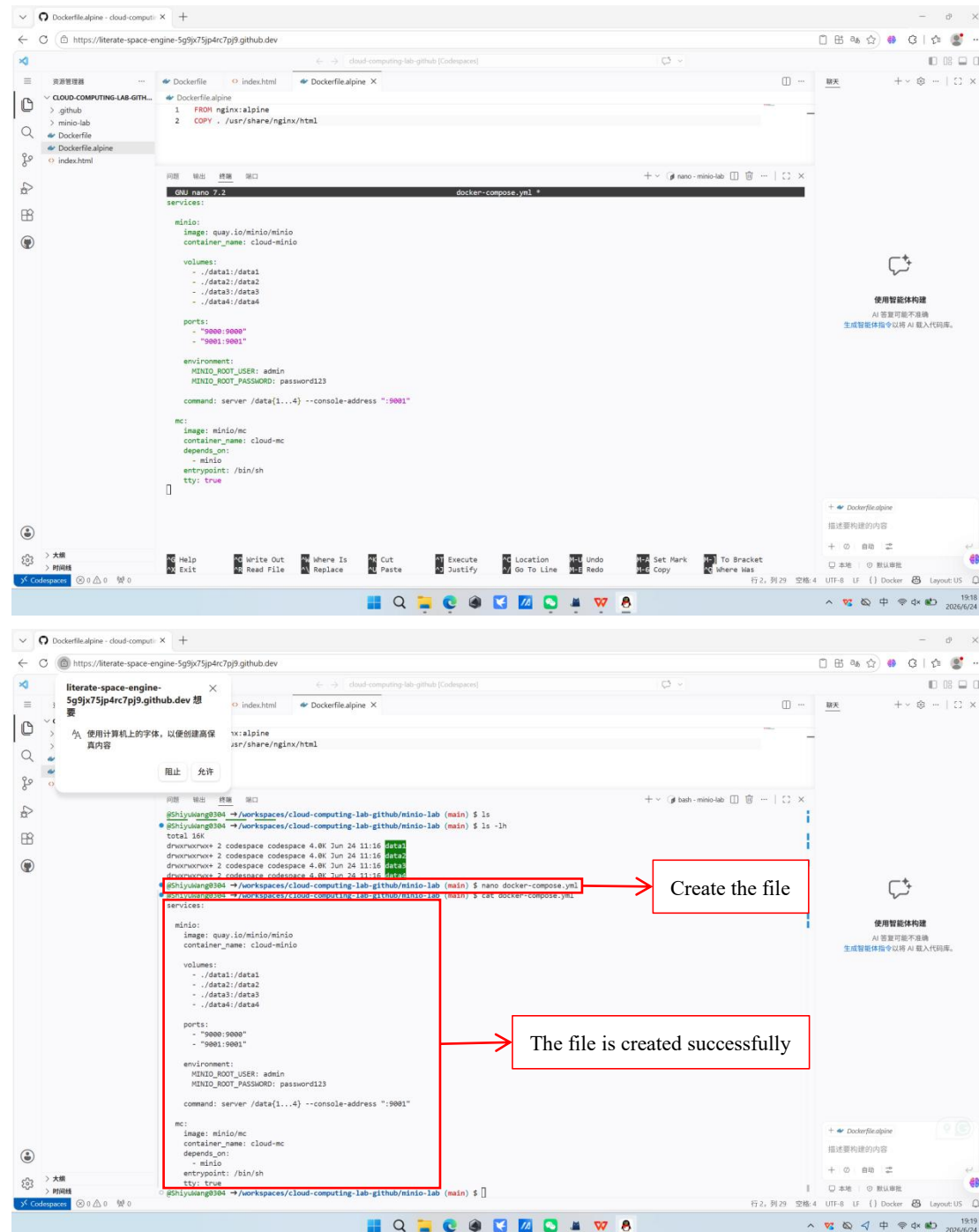
The output showed the four storage directories: `data1 data2 data3 data4`



These four directories were later mounted into the MinIO container as distributed storage paths.

3. Step 3: Creating the docker-compose.yml File

I created a docker-compose.yml file inside the minio-lab directory. This file defines two services: the MinIO server and the MinIO client.



This configuration launches a MinIO distributed storage service using four mounted directories and also starts a MinIO Client container for management operations.

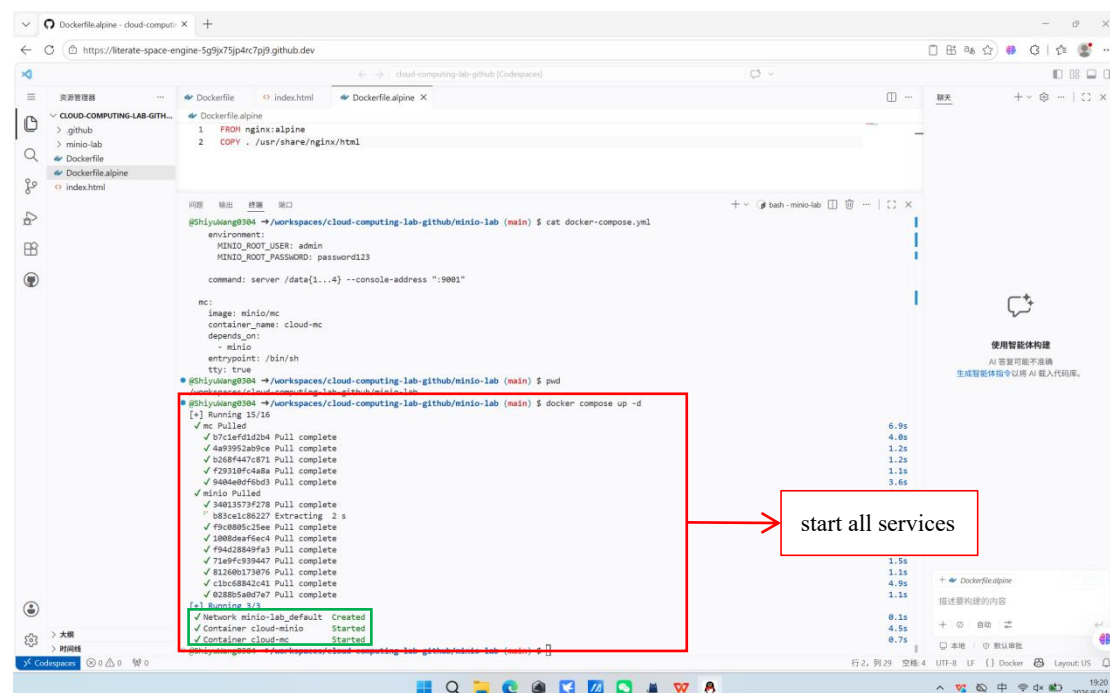
4. Step 4: Explanation of the Docker Compose File

The main components of the docker-compose.yml file are explained below.

Component	Explanation
image	Specifies the Docker image used to create the container. The MinIO server uses quay.io/minio/minio, and the client uses minio/mc.
container_name	Defines a fixed name for each container. In this experiment, the containers are named cloud-minio and cloud-mc.
volumes	Mounts local directories into the container. The directories data1, data2, data3, and data4 simulate four independent storage disks.
ports	Maps container ports to the host. Port 9000 is used for the MinIO API, and port 9001 is used for the MinIO web console.
environment	Sets environment variables for the MinIO service, including the root username and password.
command	Starts the MinIO server with four storage directories and sets the console address to port 9001.
depends_on	Ensures that the MinIO server container starts before the MinIO client container.

5. Step 5: Launching the Distributed MinIO Cluster

After creating the configuration file, I started all services using Docker Compose: docker compose up -d.



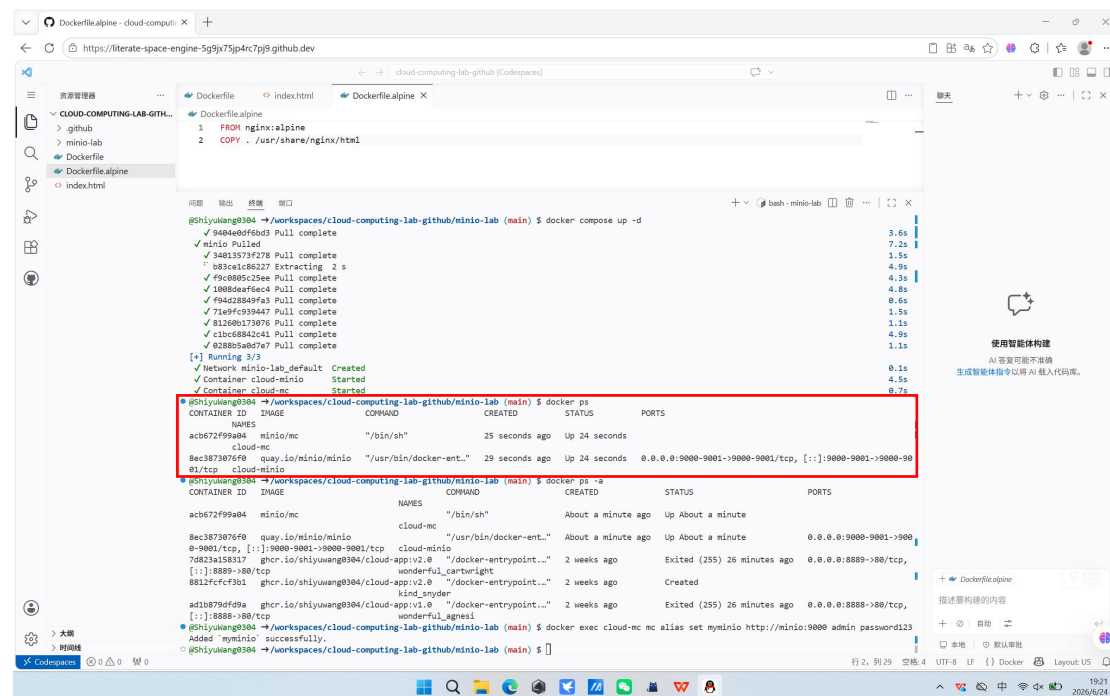
```
bash - minio-lab
@ShiyuKuang3904 ➔ /workspaces/cloud-computing-lab-github/minio-lab (main) $ cat docker-compose.yml
environment:
  MINIO_ROOT_USER: admin
  MINIO_ROOT_PASSWORD: password123
command: server /data[1..4] --console-address ":9001"

mc:
  image: minio/mc
  container_name: cloud-mc
  depends_on:
    - minio
  entrypoint: /bin/sh
  tty: true

@ShiyuKuang3904 ➔ /workspaces/cloud-computing-lab-github/minio-lab (main) $ pwd
/workspaces/cloud-computing-lab-github/minio-lab

@ShiyuKuang3904 ➔ /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose up -d
[*] Running 15/16
✓ mc Pulled
✓ 87c1af8fd1284 Pull complete
✓ 4a93952a09ce Pull complete
✓ b268f447c871 Pull complete
✓ f29318fc44da Pull complete
✓ 9a0a0ef8b0d3 Pull complete
✓ minio Pulled
✓ 34013573f278 Pull complete
✓ b83c1c616227 Extracting 2 s
✓ f9c88095c25ee Pull complete
✓ 1008deaf6ec4 Pull complete
✓ f94c285a99a3 Pull complete
✓ 71e9fc939447 Pull complete
✓ 81260b173076 Pull complete
✓ c1bc884241 Pull complete
✓ 628805ab4747 Pull complete
[*] Running 3/3
✓ Network minio-lab_default Created
✓ Container cloud-minio Started
✓ Container cloud-mc Started
```

Then, I checked the running containers: docker ps



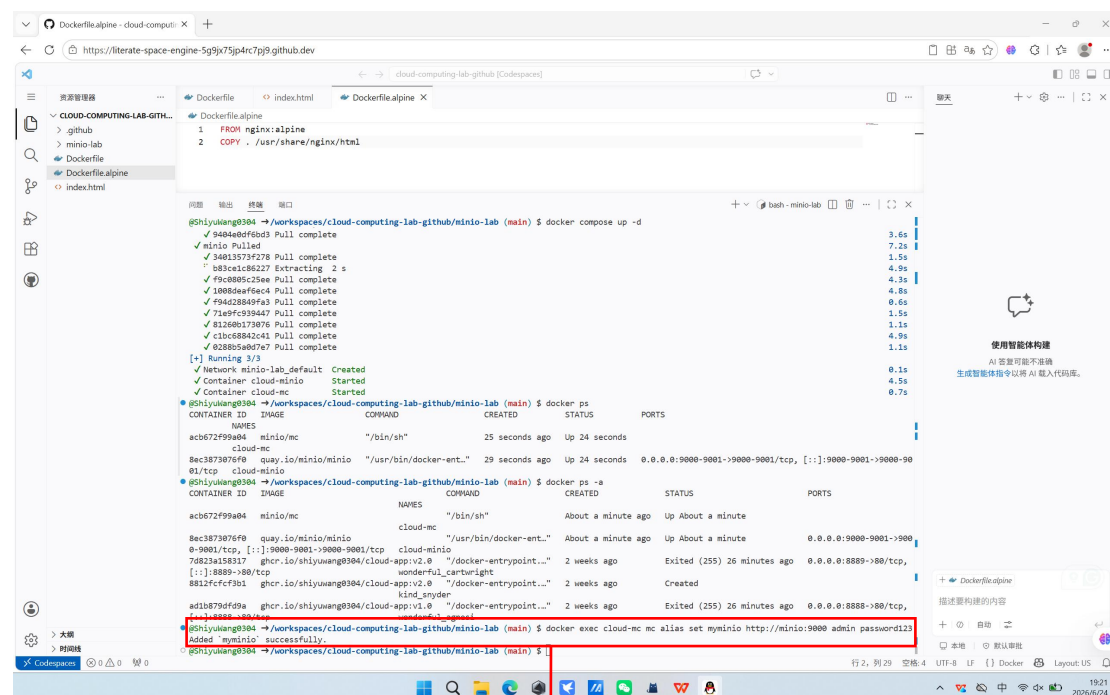
```
gShiyuang8304 → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose up -d
✓ 9404e0f6d3 Pull complete
✓ minio Pulled
✓ 34013573f278 Pull complete
  b83c1c86227 Extracting 2 s
  ✓ 49c8085c25ee Pull complete
  ✓ 10808a5f6ec4 Pull complete
  ✓ f94d28849fa3 Pull complete
  ✓ 71e9fc939447 Pull complete
  ✓ 81260b173076 Pull complete
  ✓ c1bc68842c41 Pull complete
  ✓ 6288b5a8d7e7 Pull complete
[+] Running 3/3
✓ Network minio-lab_default Created
✓ Container cloud-minio Started
✓ Container cloud-mc Started

gShiyuang8304 → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
ac6b72f99a04   minio/mc   "/bin/sh"               25 seconds ago Up 24 seconds
8ec3873076f0   quay.io/minio/minio   "/usr/bin/docker-ent..." 29 seconds ago Up 24 seconds   0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp
gShiyuang8304 → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
ac6b72f99a04   minio/mc   "/bin/sh"               About a minute ago Up About a minute
8ec3873076f0   quay.io/minio/minio   "/usr/bin/docker-ent..." About a minute ago Up About a minute   0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp
7d623a158117   ghcr.io/shiyuang8304/cloud-app:v2.0   "/docker-entrypoint..." 2 weeks ago   Exited (255) 26 minutes ago   0.0.0.0:8889->88/tcp, [::]:8889->88/tcp
8812f2cfc701   ghcr.io/shiyuang8304/cloud-app:v2.0   "/docker-entrypoint..." 2 weeks ago   Created
ad1b879d9f0a   ghcr.io/shiyuang8304/cloud-app:v1.0   "/docker-entrypoint..." 2 weeks ago   Exited (255) 26 minutes ago   0.0.0.0:8888->88/tcp, [::]:8888->88/tcp
gShiyuang8304 → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
Added 'myminio' successfully.
gShiyuang8304 → /workspaces/cloud-computing-lab-github/minio-lab (main) $
```

The output showed that both cloud-minio and cloud-mc were running successfully. This confirmed that the distributed MinIO cluster had been launched.

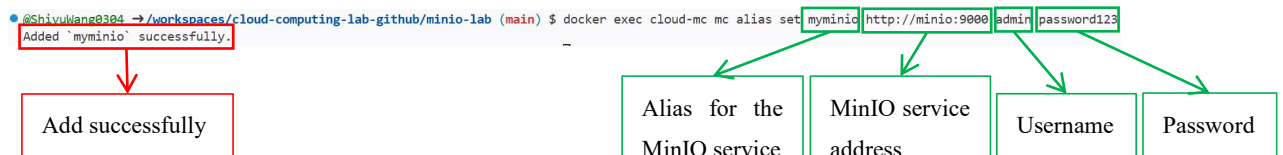
6. Step 6: Configuring the MinIO Client Alias

To allow the MinIO Client container to communicate with the MinIO server, I configured an alias named myminio: docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123



```
gShiyuang8304 → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose up -d
✓ 9404e0f6d3 Pull complete
✓ minio Pulled
✓ 34013573f278 Pull complete
  b83c1c86227 Extracting 2 s
  ✓ 49c8085c25ee Pull complete
  ✓ 10808a5f6ec4 Pull complete
  ✓ f94d28849fa3 Pull complete
  ✓ 71e9fc939447 Pull complete
  ✓ 81260b173076 Pull complete
  ✓ c1bc68842c41 Pull complete
  ✓ 6288b5a8d7e7 Pull complete
[+] Running 3/3
✓ Network minio-lab_default Created
✓ Container cloud-minio Started
✓ Container cloud-mc Started

gShiyuang8304 → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
ac6b72f99a04   minio/mc   "/bin/sh"               25 seconds ago Up 24 seconds
8ec3873076f0   quay.io/minio/minio   "/usr/bin/docker-ent..." 29 seconds ago Up 24 seconds   0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp
gShiyuang8304 → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
ac6b72f99a04   minio/mc   "/bin/sh"               About a minute ago Up About a minute
8ec3873076f0   quay.io/minio/minio   "/usr/bin/docker-ent..." About a minute ago Up About a minute   0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp
7d623a158117   ghcr.io/shiyuang8304/cloud-app:v2.0   "/docker-entrypoint..." 2 weeks ago   Exited (255) 26 minutes ago   0.0.0.0:8889->88/tcp, [::]:8889->88/tcp
8812f2cfc701   ghcr.io/shiyuang8304/cloud-app:v2.0   "/docker-entrypoint..." 2 weeks ago   Created
ad1b879d9f0a   ghcr.io/shiyuang8304/cloud-app:v1.0   "/docker-entrypoint..." 2 weeks ago   Exited (255) 26 minutes ago   0.0.0.0:8888->88/tcp, [::]:8888->88/tcp
gShiyuang8304 → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
Added 'myminio' successfully.
gShiyuang8304 → /workspaces/cloud-computing-lab-github/minio-lab (main) $
```

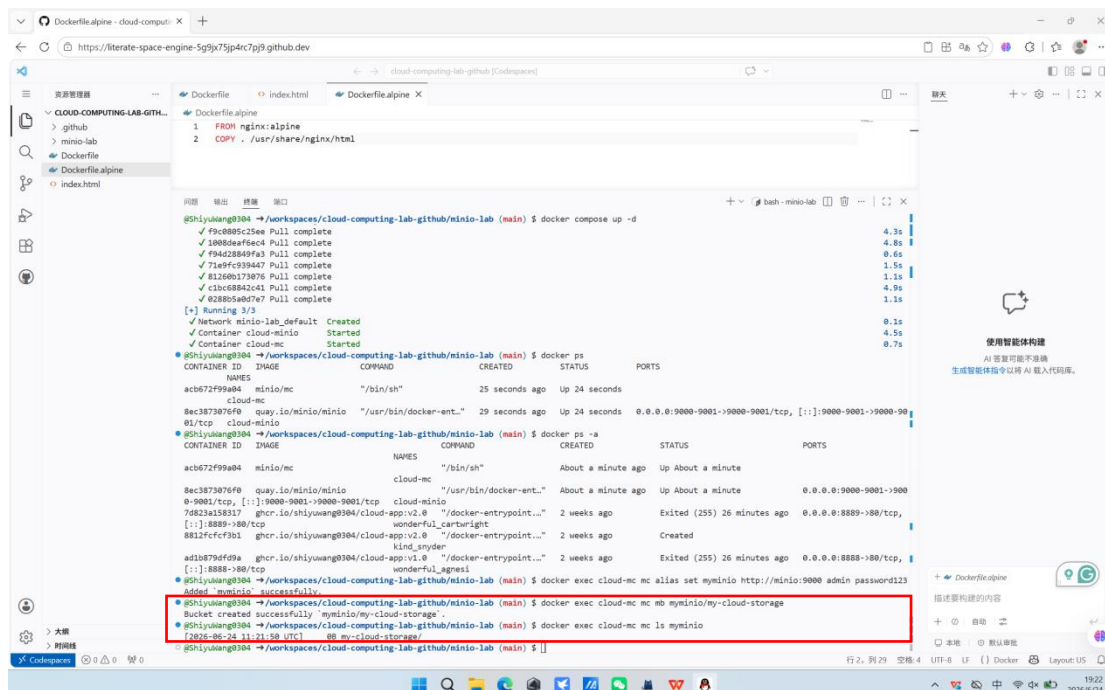


The command returned a success message indicating that the alias was added successfully. This alias was then used in later commands to access the MinIO server.

7. Step 7: Creating a Bucket

I created a bucket named my-cloud-storage using the MinIO Client: `docker exec cloud-mc mc mb myminio/my-cloud-storage`

Then, I verified the bucket by listing all buckets under the MinIO server: `docker exec cloud-mc mc ls myminio`



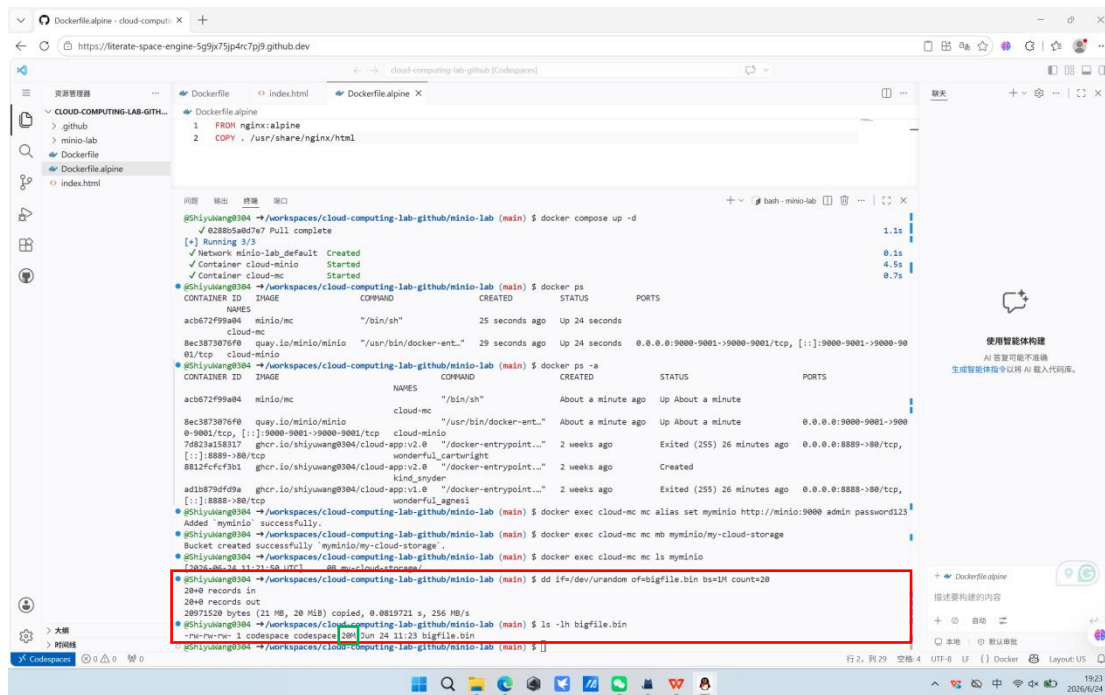
The bucket my-cloud-storage was displayed, which confirmed that the bucket had been created successfully.

8. Step 8: Creating and Uploading a Large Test File

To test distributed object storage, I created a 20 MB random binary file: `dd if=/dev/urandom of=bigfile.bin bs=1M count=20`

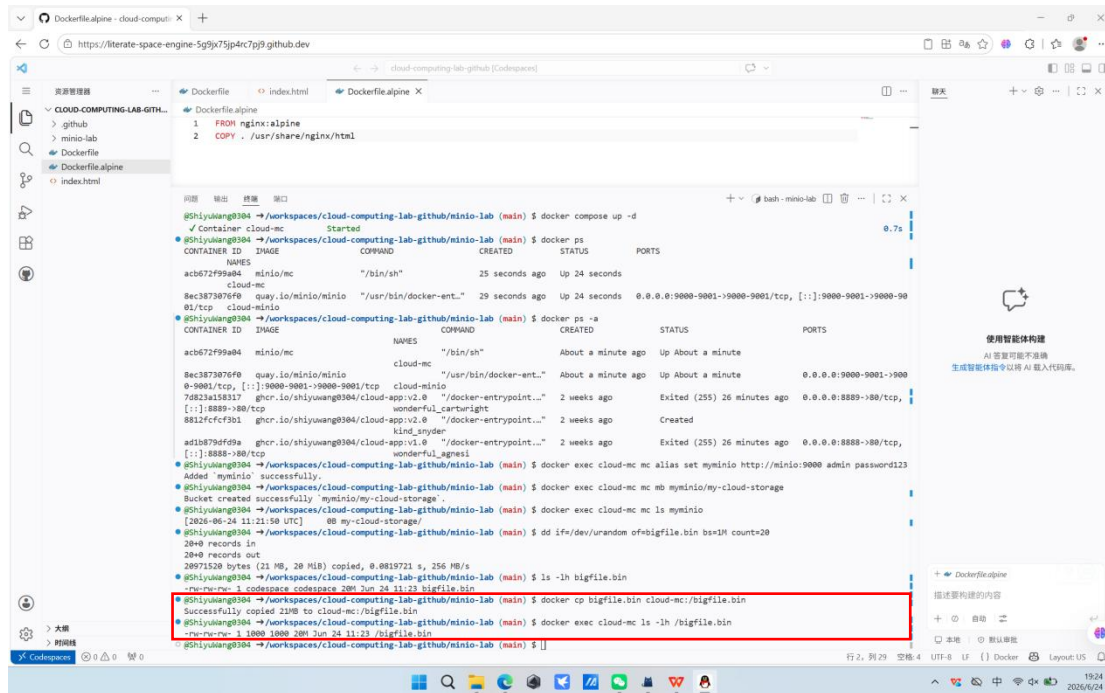
Then, I checked the file size: `ls -lh bigfile.bin`

The output showed that bigfile.bin was approximately 20 MB.

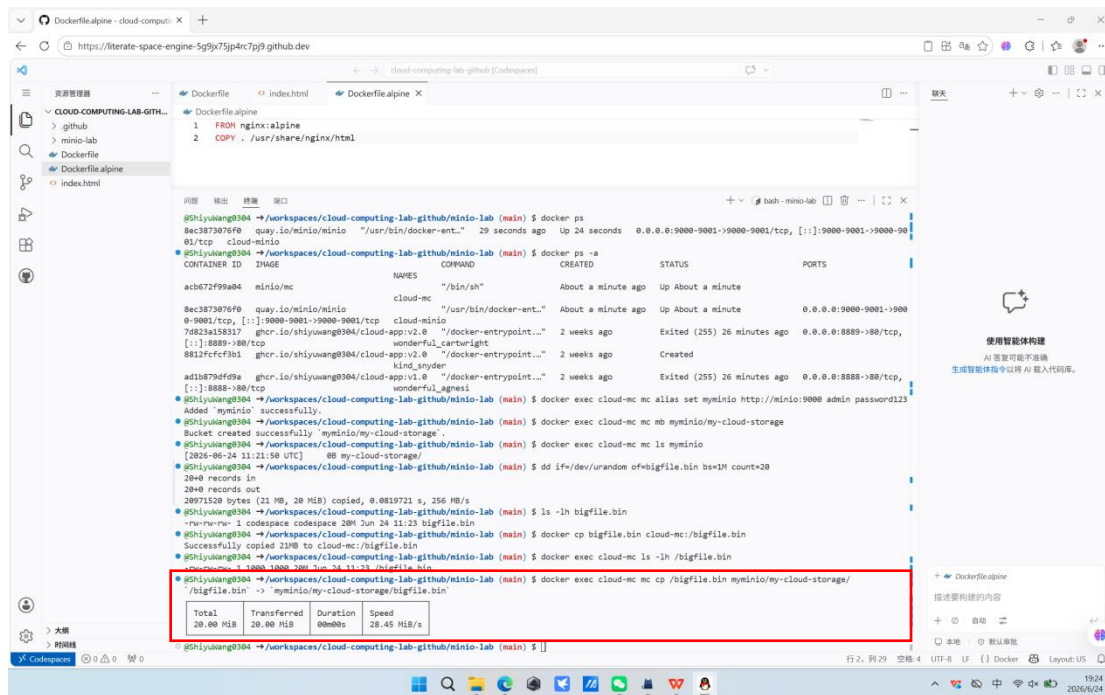


Next, I copied the file into the MinIO Client container: `docker cp bigfile.bin cloud-mc:/bigfile.bin`

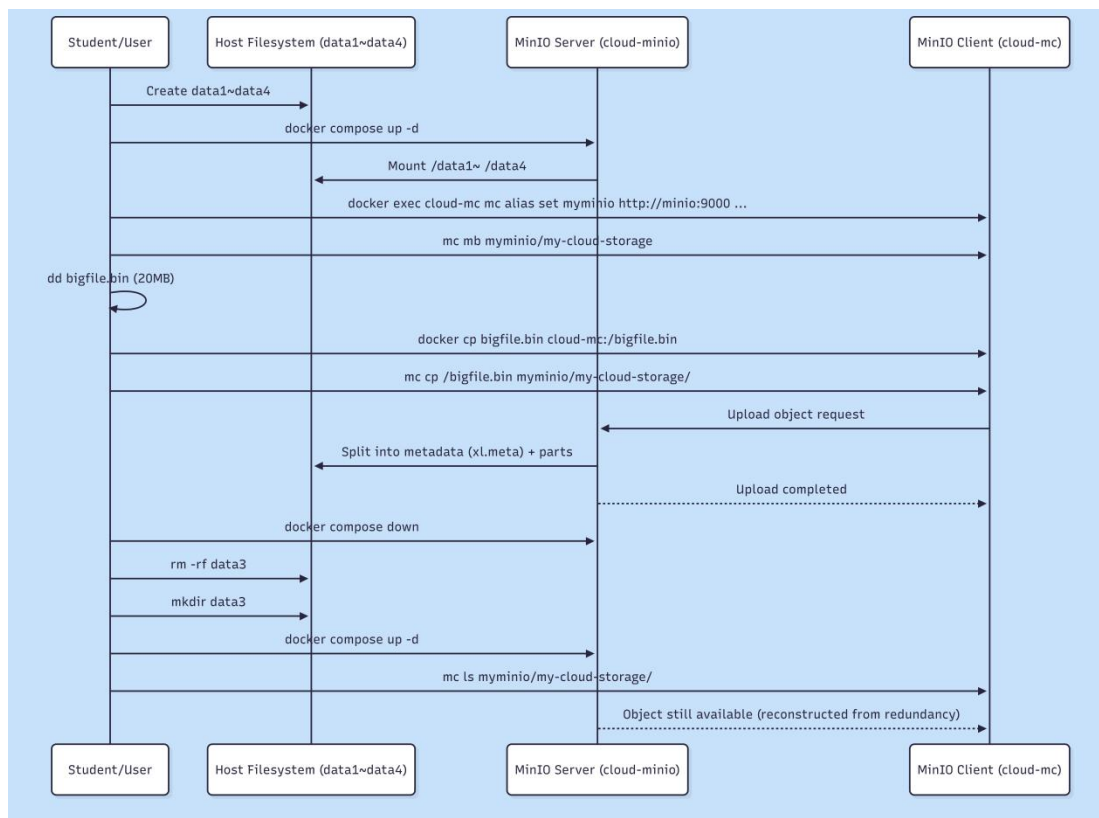
I verified that the file existed inside the container: `docker exec cloud-mc ls -lh /bigfile.bin`



Finally, I uploaded the file to the bucket: `docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/`



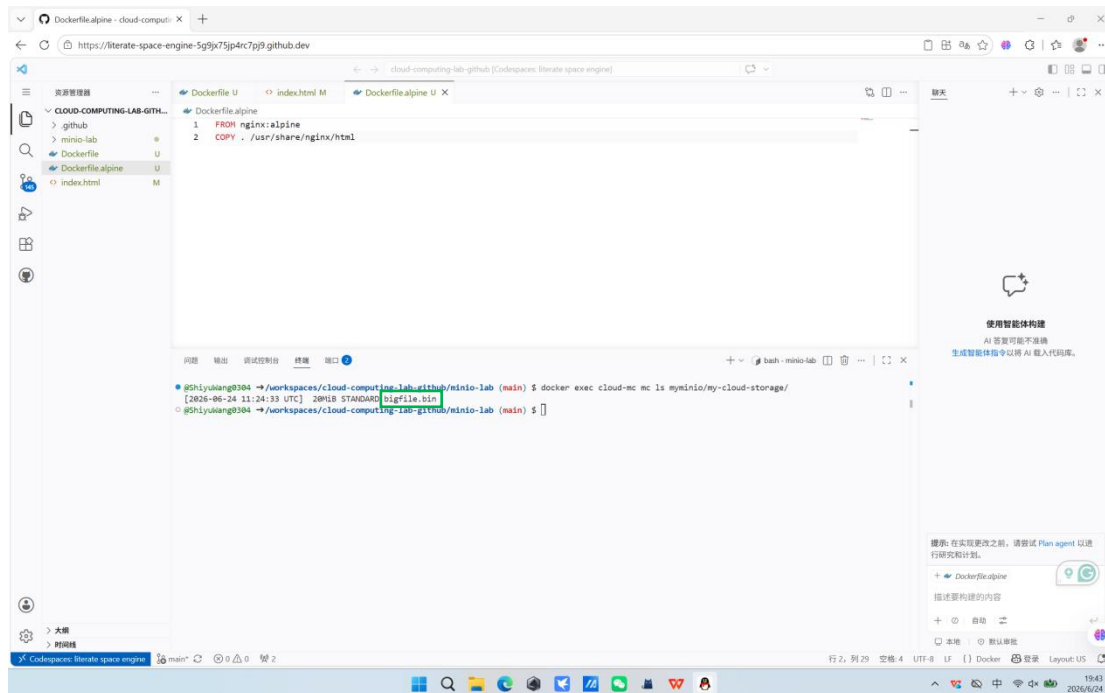
The upload progress was displayed in the terminal, indicating that the object was successfully uploaded to MinIO.



9. Step 9: Verifying the Uploaded Object

After uploading the file, I listed the objects in the bucket: `docker exec cloud-mc mc ls myminio/my-cloud-storage/`

The output showed: bigfile.bin



This confirmed that the file had been successfully uploaded as an object in the bucket.

10. Step 10: Observing Backend Storage Fragmentation

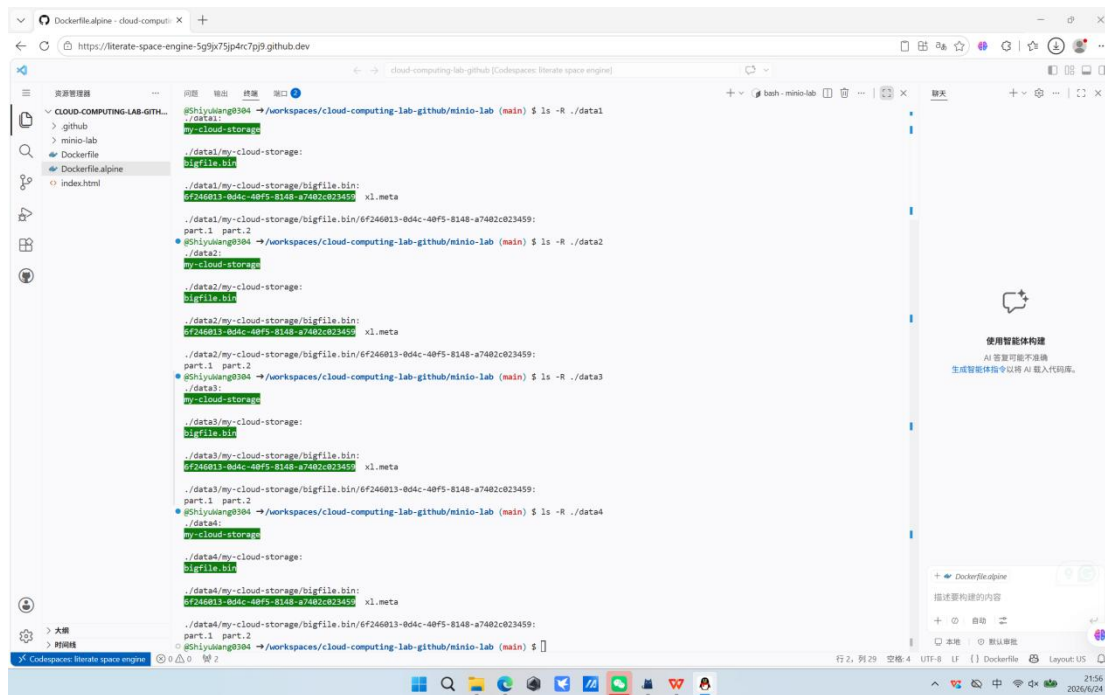
To observe how MinIO stores the uploaded object in the backend storage directories, I checked the contents of each simulated disk:

```
ls -R ./data1
```

```
ls -R ./data2
```

```
ls -R ./data3
```

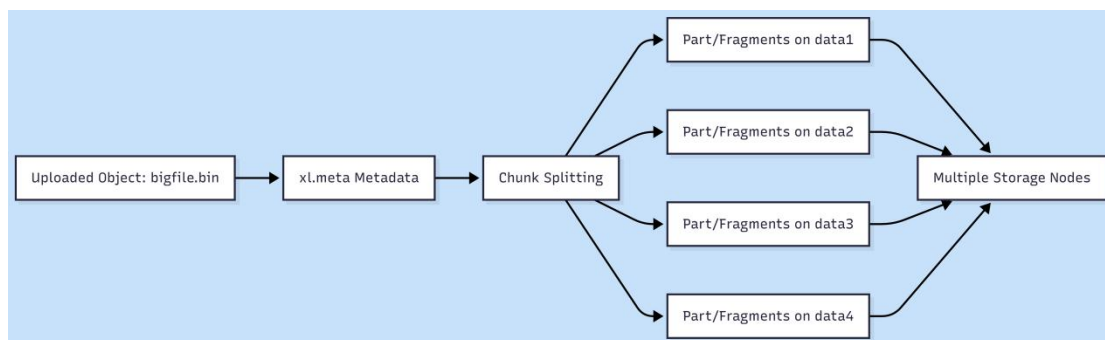
```
ls -R ./data4
```



The results showed that the object data was distributed across multiple storage directories. Each directory contained MinIO internal storage structures, including object metadata such as xl.meta and related object data parts.

This demonstrated that the uploaded object was not stored as a single ordinary file in one directory. Instead, MinIO split and distributed the object across the four storage directories using its distributed object storage mechanism.

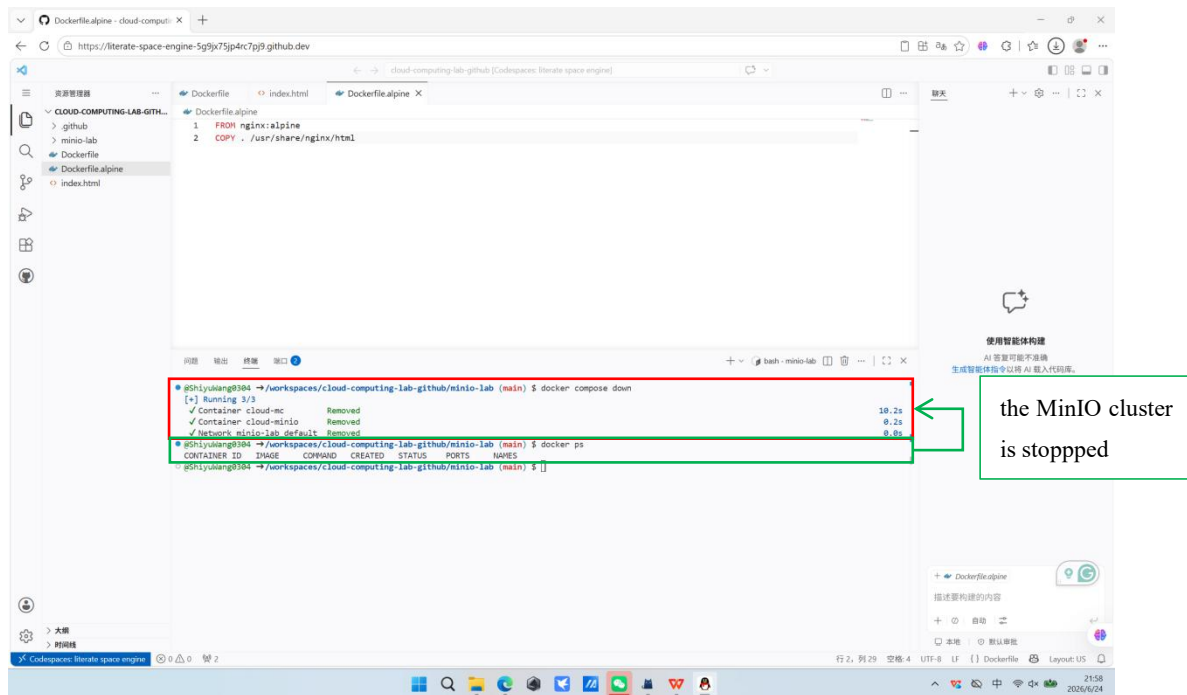
The observed storage process can be summarized as follows.



By comparing the structures of data1, data2, data3, and data4, I confirmed that MinIO uses metadata and distributed fragments to manage object storage.

11. Step 11: Simulating Disk Failure

To simulate a storage disk failure, I first stopped the MinIO cluster: docker compose down



Then, I deleted the data3 directory: `sudo rm -rf data3`

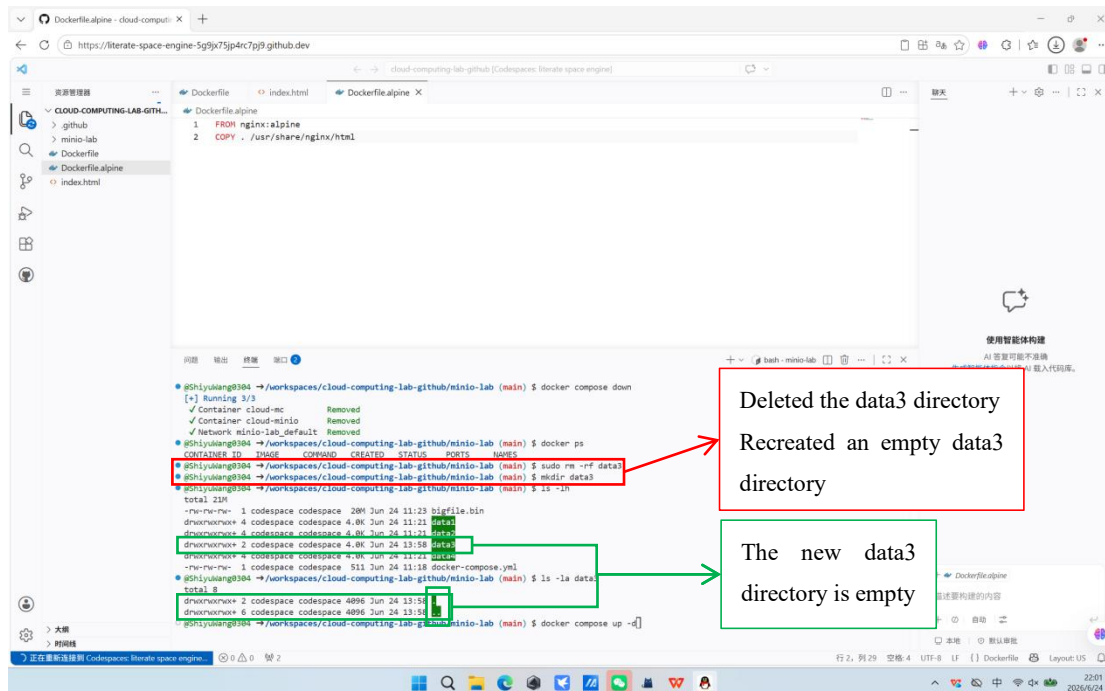
After that, I recreated an empty data3 directory: `mkdir data3`

I verified the directory status:

`ls -lh`

`ls -la data3`

The new data3 directory was empty, which represented a replaced empty disk after a disk failure.



Then, I restarted the MinIO cluster: `docker compose up -d`

Finally, I checked the running containers: `docker ps`

The screenshot shows a VS Code editor with a Dockerfile for a multi-stage build. The Dockerfile has two stages: 'FROM nginx:alpine' and 'COPY ./usr/share/nginx/html'. The terminal output shows the execution of 'docker compose up' and 'docker ps' commands. The 'docker ps' output is highlighted by a red box, showing the container 'cloud-mc' running on 'minio-lab'.

```
Dockerfile
# Dockerfile.alpine
1 FROM nginx:alpine
2 COPY ./usr/share/nginx/html
```

```
bash - minio-lab
$ docker compose up -d
[*] Running 2/2
✓ Container cloud-minio Started 0.4s
✓ Container cloud-mc Started 0.2s
$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS      PORTS
9c8b891cb0b0   minio/mc   "/bin/sh"                2 hours ago Up 18 seconds
cloud-mc
23f9fe185904   quay.io/minio/minio   "/usr/bin/docker-ent..." 2 hours ago Up 18 seconds   0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp
cloud-minio
$
```

After restarting the cluster with one storage directory emptied, I configured the MinIO Client alias again: `docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123`

Then, I listed the objects in the bucket: `docker exec cloud-mc mc ls myminio/my-cloud-storage/`

The object `bigfile.bin` was still visible in the bucket. This showed that the object remained accessible even after one simulated disk was deleted.

The screenshot displays the VS Code editor with a Dockerfile and a terminal window. The Dockerfile is named 'Dockerfile.alpine' and contains the following content:

```

1 FROM nginx:alpine
2 COPY . /usr/share/nginx/html

```

The terminal window shows the following commands and output:

```

* gshiyuang@304 → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc ls myminio/my-cloud-storage/
[2026-06-24 11:24:33 UTC] 20MB STANDARD bigfile.bin

* gshiyuang@304 → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc cp myminio/my-cloud-storage/bigfile.bin /recovered-bigfile.bin
myminio/my-cloud-storage/bigfile.bin' -> '/recovered-bigfile.bin'

Total    Transferred    Duration    Speed
20.00 MiB    20.00 MiB    00m01s    10.23 MiB/s

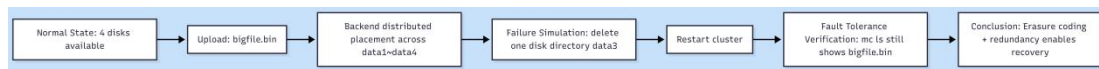
* gshiyuang@304 → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc ls -lh /recovered-bigfile.bin
-rw-r--r-- 1 root root 20M Jun 24 16:56 /recovered-bigfile.bin

* gshiyuang@304 → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ]

```

The output of the 'ls' command is highlighted with a red box, showing a table with the following data:

Total	Transferred	Duration	Speed
20.00 MiB	20.00 MiB	00m01s	10.23 MiB/s



This result demonstrates that MinIO uses erasure coding and distributed redundancy to tolerate disk failures. Even when one storage disk loses its data, the object can still be recovered and accessed from the remaining disks.

五、Reflection

1. Problems Encountered and Proposed Solutions

Problem Encountered	Impact / Observation	Proposed Solution
MinIO Client alias not found after restarting containers	After restarting the MinIO cluster, the mc client no longer recognized the previously configured alias (e.g., myminio), causing commands like mc ls to fail.	Reconfigure the alias after every restart using: <code>docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123</code> .
Unexpected “object not found / folder not found” when verifying the bucket	The verification command referred to an alias/path that was not available inside the cloud-mc container at that time.	Confirm the correct bucket name and path with mc ls after ensuring the alias is set, then re-run the upload and verification steps.

2. Explain in 3–5 sentences why data is still available after deleting one storage directory.

MinIO uses erasure coding to split and encode an object into multiple fragments across the distributed storage directories (data1 – data4). Because redundancy is built into the erasure-coded fragments, the system can reconstruct the original object even if one storage directory (e.g., data3) is missing or cleared. Therefore, after deleting one directory and restarting the cluster, mc ls can still show the object and MinIO can recover its content from the remaining fragments. This confirms that the distributed setup is resilient to single-disk failure.